

AI Tutor App — Design Brief

Oct 1, 2026 · @Evan Perez

The problem and the pitch

General AI chat already explains well, but it forgets, it describes instead of shows, and it can't see the whole course. This app is an AI tutor that remembers what you struggled with, shows it visually at the moment of confusion, and maps everything to your actual curriculum.

Two observations drive it:

- **Students often use AI in ways that skip the learning.** Chat answers are easy to copy. An app that teaches through interactive visuals, one concept at a time, is hard to misuse that way.
- **Explanations are mostly words.** Quantitative subjects like stats and econ are shapes and relationships. A graph you can drag teaches in seconds what a page of text struggles to.

Inspired by Lingua's on-the-spot generated lessons, with one key difference: this app doesn't push you through a generated curriculum. It's a tutor that organizes everything you ask, over time, into a map of what you know.

Who it's for

Evan is the first user, starting with his own stats and microeconomics courses. Once it works for him, it expands in two directions:

- **Students and self-learners (consumer).** Pays a small subscription, decides in minutes, wants it to save time.
- **Schools (institutional).** Buys through administrators over months, cares about data privacy, teacher visibility, and a tool that teaches instead of doing homework.

The sequence is: prove it works for Evan, then for students like him, then use that proof to pitch schools. Schools may be the stronger buyer, because they want the "teach, don't solve" rule that some students will resist.

How Evan studies today

Evan’s real Claude conversations fall into four jobs, and the app has to beat general chat at all four, not just concept explanations.

Job	Example chats	Where general chat falls short	What the app does instead
Exam and quiz prep	Chapter 7 quiz study, Microeconomics midterm prep, AQM concepts study guide	Rebuilds a study guide from scratch each time. Evan had to ask "did we cover everything?"	Lights up the concepts on the exam, shows weak and faded ones, builds a review session starting with the weakest
Homework help	Chapter 6 problems, Sampling distribution calculation help	Fixed an empirical-rule error with more numbers, when the mistake was visual	Hint ladder, plus a visual triggered at the moment of the mistake; each problem strengthens or flags its concepts
Understanding a concept	Understanding z scores, Supply curve slope characteristics	Good Socratic hints, but all in words	Interactive lesson with sliders, one concept at a time
Applying it to something real	Gambling game variability analysis	Reasoned it out in text	Turns curiosity into a simulation, connected back to the concepts it uses

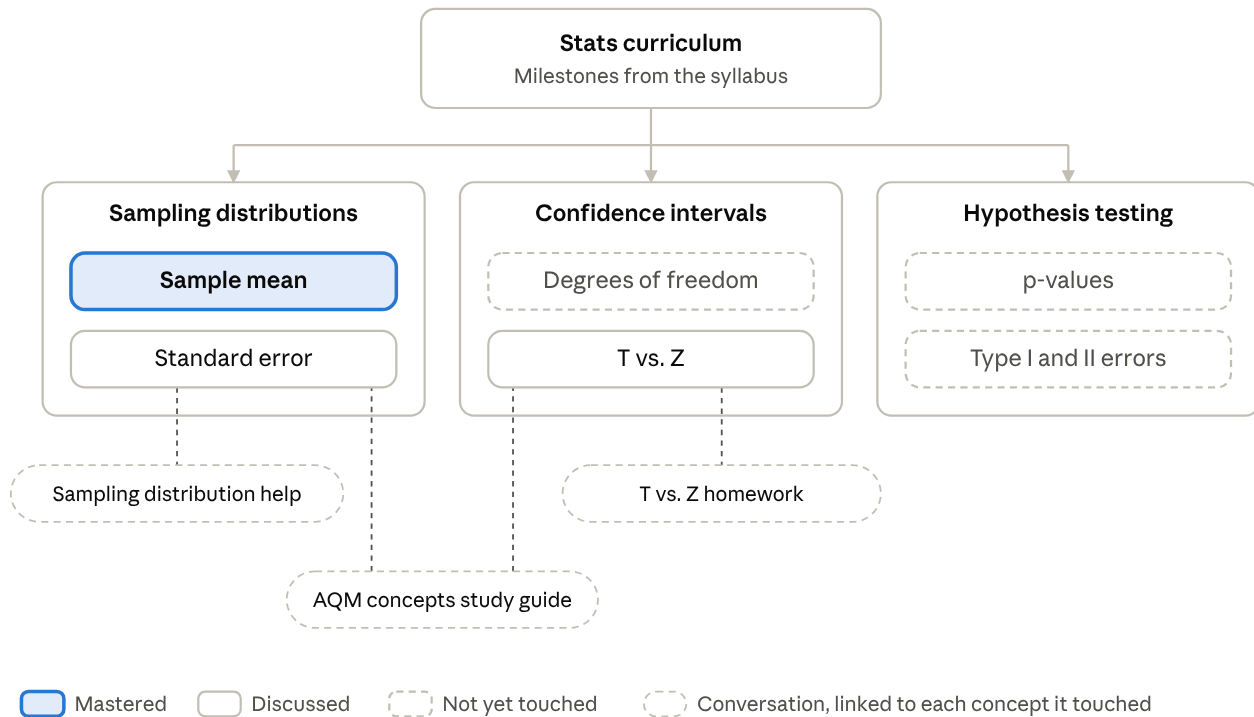
What general chat already does well, and the app must keep: one question at a time, hints before answers, catching the exact mistake, practice quizzes, and honest readiness checks.

Core structure

The map is built from concepts, not conversations, because conversations don't map cleanly to ideas. One late-night chat can cover three concepts, and three chats can circle the same confusion.

- **Curriculum:** one per course or learning goal.
- **Milestones:** the big units, from the syllabus or a goal.
- **Concepts:** the individual ideas inside each milestone. These are the nodes on the map, and they're what gets mastered.
- **Conversations:** attached to every concept they touched, like receipts. Click a concept to see each conversation where it came up.

Concepts are what you master; conversations attach to them



structure · curriculum, milestones, concepts, conversations

The AI's filing job is simple sorting: "which concepts did this conversation touch?" Prerequisite links between concepts create branches, and you can always move a conversation that got filed wrong.

The three screens

The app has three screens, and the map is home, not a chat sidebar. Sidebars sort by time, so threads sink and get lost. The map sorts by meaning, and the AI does the filing.

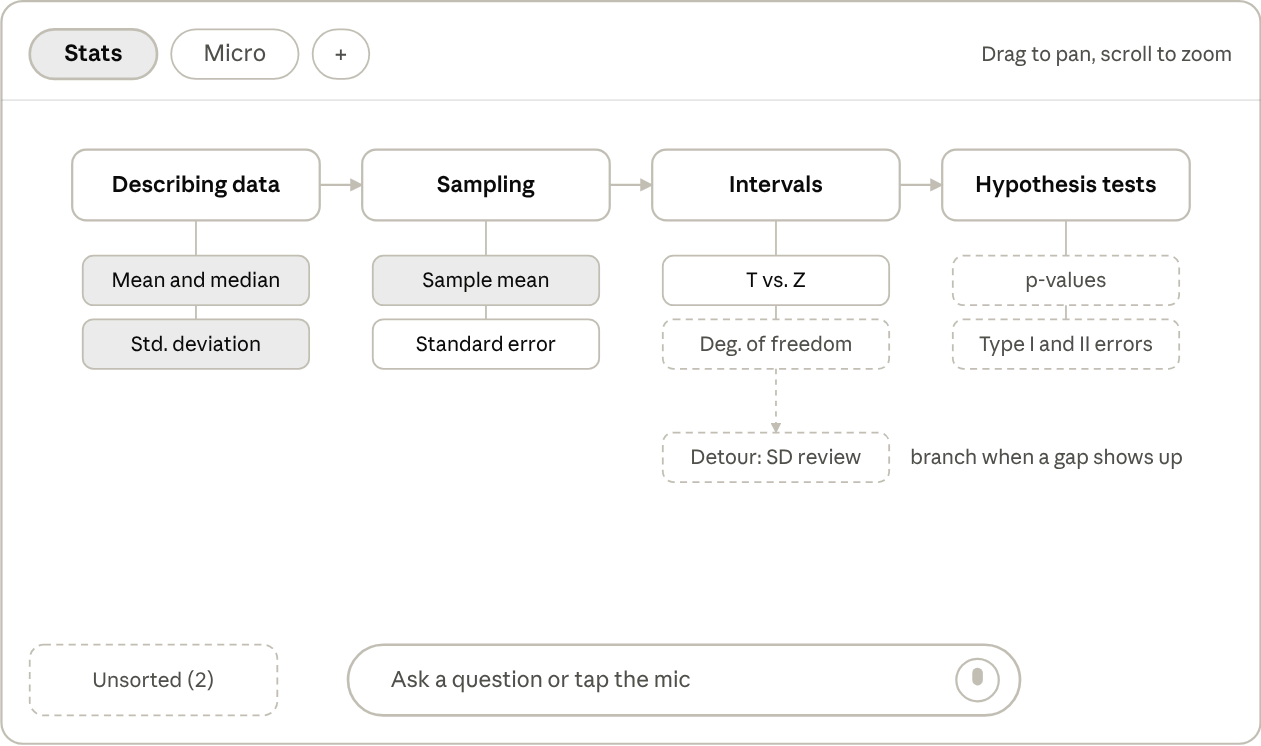
1. **Map (home).** A whiteboard you can drag around, one curriculum per course. Milestones and concepts are laid out; concepts change state as you learn. A text bar sits at the bottom center for typing or speaking. You can also click a concept and ask from there, so the session starts already knowing the topic.
2. **Session.** A familiar back-and-forth, paced one concept at a time. A back-to-map button sits in the top left. A small label at the top shows where the conversation is being filed ("Filed under: Confidence intervals"), and you can correct it.
3. **Lesson.** A full-screen, distraction-free slide deck that the session opens when a concept has a visual shape. It has its own small ask button, so asking a question never means leaving full screen.

Voice input matters: Evan often thinks out loud, so speaking into the text bar should be a first-class option, not an afterthought.

Wireframes

Layout only for now: fonts, colors, and visual style come later, once the structure works. Two of Evan's references shaped these: a left-to-right milestone flowchart (from a Canva template) for the map, and a mostly blank T vs. Z slide for the lesson. Paste the original images here as reference if useful.

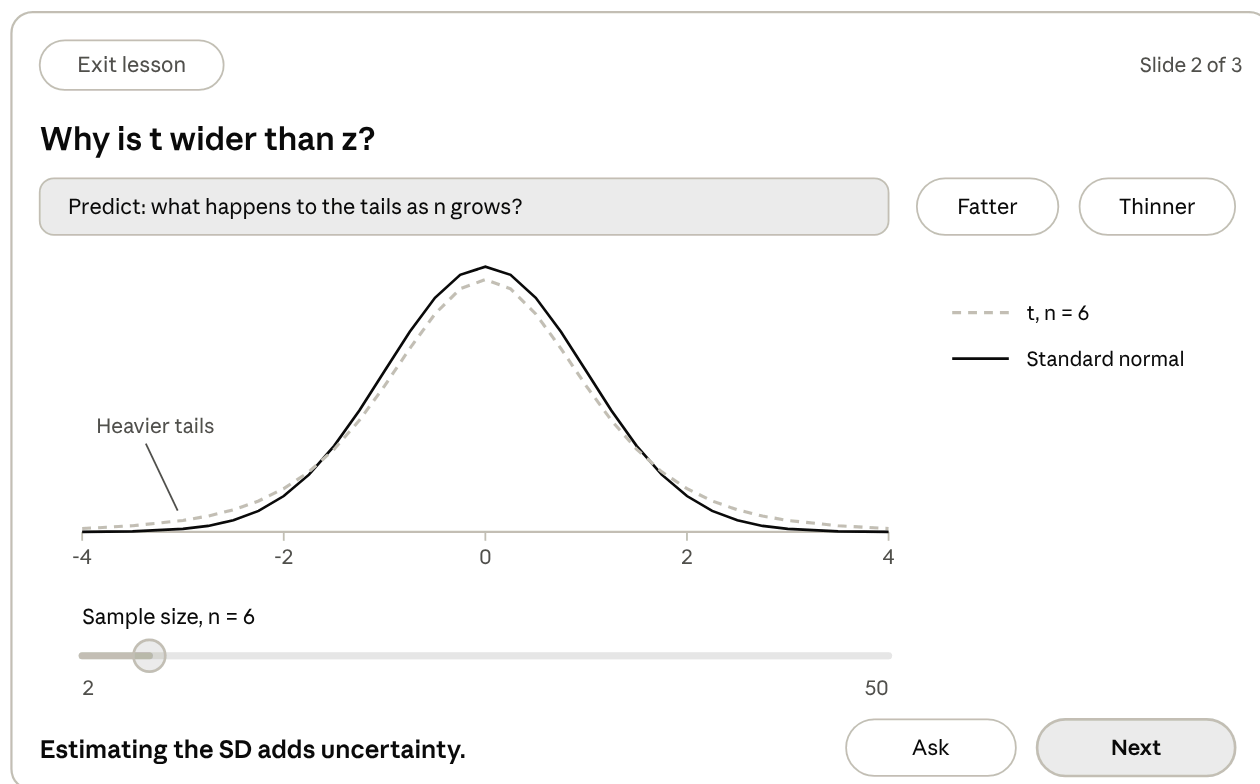
Home is a map of the course, with the ask bar always at the bottom



wireframe · map screen

Milestones run left to right like a flowchart, with concepts stacked under each. Shaded means mastered, solid means discussed, dashed means not yet touched. A detour branches off when a prerequisite gap shows up, and clicking any concept opens the conversations attached to it.

A lesson slide is one question, one interactive visual, one takeaway



wireframe · lesson slide (T vs. Z)

The slide keeps a fixed layout: question title, prediction prompt, one visual with its controls, one takeaway, and Ask and Next buttons. The session screen needs no wireframe yet; it's a familiar chat with a back-to-map button top left and the "Filed under" label at the top.

Session rules

The tutor follows concrete rules, so its behavior is predictable rather than a vibe.

The pedagogy loop. Every concept runs through the same rhythm, and the tutor advances only when you've understood, not when material runs out:

1. **Predict.** "What do you think happens to the tails if n gets bigger?" Lightweight: a one-tap guess, and skippable.
2. **Play.** Manipulate the visual: drag the slider, change the numbers.
3. **Explain.** Say what you saw in your own words.
4. **Check.** A short problem, then move to the next concept.

When to open a lesson. Open a lesson when a concept has a visual shape (a distribution, a curve, a relationship between variables) or when a mistake is visual, like mis-pairing empirical-rule bands. Stay conversational for definitions, quick clarifications, and procedure questions.

Lesson length. A short deck of three to five slides: setup, the interactive moment, then a check.

The hint ladder (teach, don't solve). The app can see your homework, so it needs a firm rule: it uses assignments to know what to teach, never to hand over answers. When you ask for help on a problem:

- 1. A hint pointing at the right idea.
- 2. A worked example on a similar problem with different numbers.
- 3. Walking you through your own problem step by step while you do the math.

You never get stonewalled, and you never get the answer handed over.

Prerequisite detours. If you jump to T vs. Z before you understand standard deviation, the tutor notices the gap and offers a short detour first.

Lessons and the visual building blocks

The AI never writes code on the spot. It fills out a short description of a slide, and the app renders it from a small set of reliable building blocks. That keeps every slide correct, fast, and in one consistent style.

Slide layout. Mostly blank, for focus: a title phrased as a question, one visual, one annotation, one takeaway sentence, the controls (sliders, draggable points), and a prediction prompt before you touch anything. Different series get different colors.

The building blocks. A T vs. Z slide and a supply-and-demand slide look different but share the same block underneath. Most of what a quantitative course needs fits in about six:

Block	Covers
Curves on axes with adjustable parameters	Distributions, supply and demand, utility curves, any function
Histograms and bar charts	Sample data, frequency, simulations

Block	Covers
Scatter plot with a fitted line	Regression, correlation
Number line or shaded region	Probabilities, intervals, empirical-rule bands
Step-by-step equation reveal	Derivations, worked examples
Labeled diagram	Flowcharts, timelines, cycles

If a subject needs something these can't do, add a seventh block rather than letting the AI improvise code.

The AI picks parameters; the app does the math. The AI says "t with 5 degrees of freedom, slider controls n, shade the tails." The renderer computes real distributions from formulas, so a curve can never be subtly wrong.

Example slide description:

```

block: curves-on-axes
title: Why is t wider than z?
series: t (df from slider), standard normal
slider: n, from 2 to 50, starts at 6
shade: tails beyond  $\pm 2$ 
annotation: "Heavier tails" pointing at the left tail
takeaway: Estimating the population SD adds uncertainty.
predict: "What happens to the tails as n grows?"

```

Mastery, struggle log, quizzes, and coverage

Mastery is the backbone of the map, so it needs a strict definition: false mastery is worse than none, because the map would say you're ready when you're not.

Concept states. Gray means untouched, outlined means discussed, filled means mastered. A filled concept slowly dims after a few weeks without practice and asks for a quick re-check (spaced repetition, shown visually).

What counts as mastered. Both a correct explain-back and a check problem solved without hints. You can also mark a concept mastered yourself, since you know things the app hasn't seen.

Struggle log. Every mistake and open question is saved under its concept automatically, such as "paired the 1σ band with 2σ instead of 3σ " or "unclear why a density can exceed 1." It feeds review sessions, exam prep, and reflection assignments.

Practice quiz mode. Generated quizzes in the style of the course's homework. Results write straight to the map: a missed question flags its concept, and a correct check problem later fills it in. Check answers are graded in a structured way, so they can't drift off into unrelated chat.

Coverage view. Because the app has read the course materials, it shows which concepts you've covered and which are still gray. You never have to ask "did we cover everything?"

Exam prep. You say "my midterm covers chapters 1–3." The map highlights those concepts, surfaces weak and faded ones, and builds a review session starting with the weakest.

AI-use export (school feature). Some courses require students to document their AI exchanges and what's still unclear. The struggle log makes that a one-click export, which is exactly what schools managing AI use want.

Course materials and the two entry modes

Each curriculum holds its own materials, uploaded once: syllabus, lecture slides, readings, problem sets, and screenshots from lecture videos. Every session already knows them, which ends re-attaching PDFs to every chat.

Grounding in materials matters for three reasons:

- The tutor uses the professor's notation, examples, and software (such as Minitab), so it never contradicts the lecture.
- It can flag conflicts, like a homework's numbers not matching the lecture video.
- It powers the coverage view and exam prep.

Later, Canvas and Google Classroom integrations pull materials in automatically. V1 uses manual upload.

The same engine has two front doors. Both draft milestones, file conversations, and track mastery; they differ only in where the skeleton comes from and whether it can change.

	School mode	Self-learner mode
Starts from	The syllabus and course materials	A stated goal ("understand stats well enough to read research papers")
Milestones	Drafted from the course's units, in the teacher's order	A light draft of five or six, clearly marked as rough
Can the skeleton change?	Fixed by the teacher	Reshapes as you learn: branches grow where your questions go, skipped milestones can be marked done
Build priority	First (it's what Evan uses)	Later

V1 scope and the first build

V1 tests the riskiest assumption first: does a paced session with interactive lessons help Evan learn faster than plain Claude? If it doesn't, the map doesn't matter. So v1 is the session and lesson screens for one course, and the map comes right after.

In v1:

- ☐ One course (Evan's stats course), materials uploaded manually
- ☐ Session screen with the pedagogy loop and hint ladder
- ☐ Lesson screen, full-screen, with an ask button
- ☐ Renderer for the curves-on-axes block, computing real distributions
- ☐ The T vs. Z deck as the first lesson
- ☐ Struggle log saved per conversation

Right after v1:

- ☐ Map as a simple vertical column of milestones with concepts under each
- ☐ Automatic filing of conversations to concepts
- ☐ Mastery states and practice quiz mode
- ☐ The remaining building blocks

Later: the draggable whiteboard, decay and spaced repetition, self-learner mode, Canvas and Google Classroom integration, voice-first input, the school features.

First build: the T vs. Z deck. Three slides:

- 1. **Setup.** "We usually don't know the population standard deviation. What changes when we estimate it from the sample?"
- 2. **Interactive moment.** A standard normal curve and a t curve overlaid, with a slider for sample size. Predict first: "What happens to the tails as n grows?" At n = 3 the t curve is squat with fat tails; as n increases it settles onto the normal curve. Takeaway: estimating the SD adds uncertainty, and that uncertainty shrinks as the sample grows.
- 3. **Check.** A short question, such as "With n = 8 and an unknown σ , do you use z or t, and why?", graded in a structured way.

The test of success: Evan uses it on his real stats homework for two weeks and judges whether he understood faster and remembered longer than with plain Claude.

Tech stack

V1 runs locally on Evan's laptop, so the only cost is Claude API credits. The stack is chosen so moving to the cloud later is a configuration change, not a rewrite.

V1 (local):

Layer	Pick	Why
App	Next.js (React), opened in the browser at localhost	Screens and a small server in one project; what AI coding tools build most reliably
AI	Claude API	Teaching, slide descriptions, filing, grading. The API key stays server-side in a <code>.env</code> file, never in browser code
Database	SQLite (one file on disk)	Free, zero setup, backed up by copying one file
Database layer	Drizzle ORM	Tables defined in code; no hand-written SQL (see below)

Layer	Pick	Why
Interactive visuals	Mafs, plus jStat	Mafs is a React library for interactive math (curves, draggable points, sliders). jStat computes real distributions, so curves are always correct
Formulas	KaTeX	Clean math notation
Course materials	PDFs sent straight to the Claude API	It reads PDFs natively; no separate parsing tool
Voice	The browser's built-in speech recognition	Free; upgrade only if accuracy becomes a problem

No more manual SQL. With Lingua, queries were hand-written and pasted into the Supabase dashboard. Here, the tables (curricula, milestones, concepts, conversations, struggle log) are defined in a TypeScript schema file. One command generates and applies the changes. Claude Code can edit the schema and run the command itself, and every change lives in the project history.

Later (public): swap SQLite for hosted Postgres (Supabase or Neon), deploy on Vercel, and add login. Drizzle sits in the middle, so the swap is mostly configuration. Free tiers cover early users. A desktop wrapper (Tauri or Electron) is optional and not needed for v1.

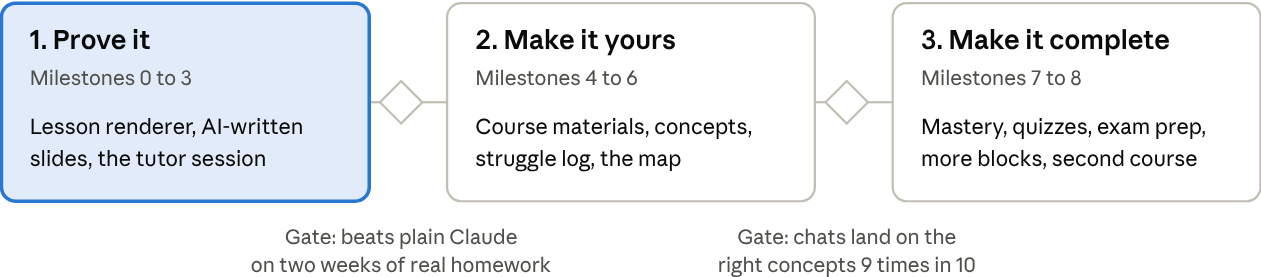
Keeping API costs down:

- **Match the model to the job.** A fast, cheap model (Haiku) for filing conversations and grading short check answers; Sonnet or Opus for teaching and writing slide descriptions.
- **Prompt caching for course materials.** The syllabus and lecture PDFs are sent with many requests; caching makes those repeated parts much cheaper.
- **Slide descriptions, not code.** The AI writes a short description per slide instead of full code, which uses far fewer tokens per lesson.

Build plan

Build the hardest, most proof-of-product piece first: an interactive lesson that actually teaches, written reliably by AI. Memory and the map only matter once that works. Each milestone has a done-when test, and nothing moves forward until it passes.

Prove the lesson works before building memory and the map



build roadmap · 3 phases, 2 gates

Phase 1 is the whole bet. If the gate after it fails, the fix is better lessons and pedagogy, not more features.

#	Milestone	What gets built	Done when
0	Setup	Next.js project, SQLite with Drizzle, API key in <code>.env</code> , one basic chat calling Claude	Evan can chat with Claude in his own app on localhost
1	Lesson renderer	The curves-on-axes block (Mafs + jStat), the slide layout, full-screen lesson screen. The T vs. Z deck written by hand as a slide description, no AI yet	The hand-written T vs. Z deck works, the slider is smooth, and the curves are mathematically correct
2	AI writes slides	Claude turns a concept into slide descriptions in the exact format the renderer expects, with validation that rejects bad ones	Asked about five different stats concepts, it produces valid, correct decks at least 9 times in 10
3	Tutor session	Session screen, the predict-play-explain-check loop, the when-to-open-a-lesson rule, the hint ladder, structured grading of check answers	Evan uses it on one real homework assignment and would rather use it than plain Claude

#	Milestone	What gets built	Done when
4	Course materials	Upload PDFs per course, prompt caching, tutor uses the professor's notation and examples, flags conflicts between sources	The tutor correctly references examples from Evan's own lecture slides
5	Concepts and struggle log	Concept list drafted from the syllabus, automatic filing of each conversation to concepts, struggle log saved per concept, unsorted tray	After a week of use, conversations land on the right concepts 9 times in 10
6	The map	Simple map view: milestones in a row, concepts under each, states shown, click a concept to see its conversations, back-to-map navigation	Evan opens the map before an exam and it shows what he knows at a glance
7	Mastery and practice	Mastery criteria (explain-back plus a check problem), practice quiz mode, exam prep from a chapter range, coverage view	A practice quiz updates the map, and exam prep starts with the weakest concepts
8	More blocks, second course	Histograms, scatter with fitted line, shaded regions, equation reveal; Micro as a second course with supply and demand	A supply-and-demand lesson works in Micro with no new code beyond the blocks

After milestone 8: the draggable whiteboard, mastery decay and spaced repetition, self-learner mode, better voice, Canvas and Google Classroom integration, cloud hosting with login, and the school features (AI-use export, teacher view).

Real-use checkpoint. Between milestones 3 and 4, Evan uses the app for two weeks on actual homework, keeping notes on where it beat or lost to plain Claude. Those notes decide what gets fixed before phase 2.

Open questions and risks

Open questions (decide during the build, not before):

- ☐ Exact mastery thresholds: one check problem or two?
- ☐ Decay timing: how many weeks before a mastered concept dims?
- ☐ How concepts are first drafted from a syllabus, and how granular they are
- ☐ What happens when a conversation touches concepts in two different courses
- ☐ Where off-map conversations live (an unsorted tray, for things like "how do I run this in Minitab?")
- ☐ Product name

Risks:

- **Students take the path of least resistance.** At 11pm with homework due, a student who wants answers may open ChatGPT instead. The hint ladder has to feel helpful, not restrictive.
- **It's a STEM tool first.** The building blocks shine for stats, econ, physics, and chemistry. For writing or history, the visual advantage mostly disappears. Say so plainly in the pitch.
- **Big AI labs already have study modes.** The edge isn't "it teaches instead of answers." It's the map, the course grounding, the struggle log, and mastery tracking together.
- **False mastery.** If mastery is too easy to earn, the map misleads. Strict criteria matter more than a pretty map.
- **Scope creep in vibe coding.** The whiteboard, integrations, and mastery logic together are too much for one build. Ship v1, test it on real homework, then expand.